

Conceptual Report

Multi-Document Retrieval-Augmented Generation (RAG) System

Based on the uploaded Jupyter notebook implementation using Azure OpenAI, LangChain, FAISS, TXT, and PDF knowledge sources

Project Type	Multi-document RAG assistant
Knowledge Sources	FAQ text file + Travel PDF + Hotel Booking PDF
Embedding Model	Azure OpenAI text-embedding-3-small
Vector Store	FAISS
Generation Model	Azure OpenAI gpt-5-mini-1
Framework	LangChain

Core architecture to remember: Load -> Chunk -> Embed -> Store -> Retrieve -> Generate

1. Project Context and Objective

The uploaded notebook implements a Retrieval-Augmented Generation (RAG) application. The purpose of the system is to answer user questions using information retrieved from a custom knowledge base rather than relying only on the language model's pretrained knowledge.

The project combines three knowledge sources: a text-based FAQ, a travel guide PDF, and a hotel-booking PDF. These sources are loaded into LangChain, transformed into chunks, converted into embeddings, indexed in FAISS, retrieved according to semantic similarity, and finally supplied to an Azure OpenAI chat model to generate a response.

High-level workflow

Knowledge Sources	Loaders	Chunking	Embeddings	FAISS	Retriever	LLM
FAQ + PDFs	TextLoader / PyPDFLoader	CharacterTextSplitter	text-embedding-3-small	Vector index	Relevant chunks	gpt-5-mini-1

The most important conceptual distinction is that the language model does not directly search the files. The retrieval layer searches vector representations of document chunks first, and only the most relevant retrieved context is passed to the chat model.

2. Knowledge Ingestion

The notebook uses different loaders for different file formats. The FAQ text file is read using TextLoader, while PDF files are read using PyPDFLoader. The resulting loaded content is stored as LangChain Document objects.

Conceptual loader flow

```
faq.txt -> TextLoader -> faq_documents
wanderwise_travel_guide.pdf -> PyPDFLoader -> travel_documents
staywise_hotel_booking_guide.pdf -> PyPDFLoader -> hotel_documents
```

The notebook then combines these loaded document collections into one unified list. This is important because the downstream chunking and vectorization stages operate on the loaded Document objects, not on the loader objects themselves.

Observed notebook result: 1 FAQ document, 4 travel PDF pages, 4 hotel PDF pages, for 9 loaded document objects in total.

3. Chunking Strategy

Large documents should not usually be embedded as one giant block. The notebook uses CharacterTextSplitter to divide the combined content into smaller retrieval units.

```
Configured values:
chunk_size = 1000
chunk_overlap = 100
```

Chunk size controls how much text is grouped into one retrieval unit. Chunk overlap repeats a portion of neighboring text so that context near a chunk boundary is less likely to be lost.

The notebook produced 5 chunks. These chunks, rather than the original files themselves, become the inputs to the embedding model.

Parameter	Purpose	Trade-off
-----------	---------	-----------

Chunk size	Controls the amount of information represented by each vector	Too small can lose context; too large can mix unrelated topics
Chunk overlap	Preserves context across boundaries	Too little may split meaning; too much duplicates content

4. Embeddings and Semantic Meaning

The system uses AzureOpenAIEmbeddings with the text-embedding-3-small deployment. An embedding model converts text into a high-dimensional numerical representation called a vector.

```
"Travel insurance is not automatically included."  
-> Embedding model ->  
[0.17, -0.62, 0.41, ...]
```

The exact numbers are not important to the user. What matters is that texts with similar meaning tend to produce vectors that are closer to one another in embedding space. This allows the system to perform semantic search rather than relying only on exact keyword matches.

5. FAISS Vector Store

After chunking, FAISS.from_documents(docs, embeddings) converts the chunks into embeddings and builds a searchable FAISS vector index. FAISS is responsible for similarity search. It does not generate answers.

Chunk	Embedding	Stored in FAISS
Chunk 1	Vector 1	Yes
Chunk 2	Vector 2	Yes
...	...	...
Chunk 5	Vector 5	Yes

6. Retrieval Process

The FAISS index is converted into a retriever. When the user asks a question, the question is embedded into the same vector space and compared with the stored chunk vectors. The retriever returns the most semantically similar chunks.

```
User question -> Query embedding -> FAISS similarity search -> Top matching chunks
```

The notebook tests retrieval with the question "what are the best hotels?". The returned content is mostly travel-guide material rather than strongly hotel-specific material. This is an important experimental result: a technically working vector store can still have imperfect retrieval relevance.

Possible causes of weaker retrieval

- The question is broad and underspecified.
- Chunks may contain several topics at the same time.
- The hotel content may not be represented strongly enough in the selected top results.
- Chunk size, overlap, and retrieval count (k) may need tuning.
- More specific entity-based questions usually produce stronger semantic matches.

7. Generation Layer

The chat generation layer uses AzureChatOpenAI with the gpt-5-mini-1 deployment. Its responsibility is different from the embedding model. The embedding model represents and retrieves meaning; the chat model generates a natural-language response from the retrieved context and the user's question.

Component	Role
Embedding model	Text -> vector representation
FAISS	Vector similarity search
Retriever	Returns relevant document chunks
Chat model	Question + retrieved context -> answer

8. RAG Chain

The notebook uses `RetrievalQA.from_chain_type(...)` to connect the retriever and the Azure chat model. The option `return_source_documents=True` is important because it allows the system to return the retrieved evidence along with the generated answer.

```
Question -> Retriever -> FAISS -> Relevant chunks -> LLM -> Answer + source documents
```

9. Grounding and Hallucination Observation

The notebook's final response demonstrates both the value and the limitation of RAG. The retrieved travel-insurance source mentions reviewing policy wording and keeping supporting documentation. However, the generated response expands into additional claims-related details that are not clearly present in the retrieved source.

This suggests that the model is combining retrieved information with its own pretrained knowledge. That behavior may be acceptable for a general assistant, but a strict enterprise RAG system often needs stronger grounding controls.

Recommended grounding instruction

```
Answer the question using only the supplied context. If the answer cannot be determined from the context, state that the information was not found in the provided documents. Do not add unsupported facts.
```

10. Secret and Environment Management

The notebook loads Azure configuration from environment variables using python-dotenv. This is a good design because secrets such as API keys do not need to be hard-coded in the notebook.

```
AZURE_OPENAI_API_KEY
AZURE_OPENAI_ENDPOINT
AZURE_EMBEDDING_DEPLOYMENT
AZURE_CHAT_DEPLOYMENT
```

This separation improves security and makes the project easier to move between development, testing, and production environments.

11. Strengths of the Implementation

Capability	Implementation
Multiple input formats	TXT and PDF
Multiple knowledge sources	FAQ + travel + hotel booking
Chunking	CharacterTextSplitter
Embeddings	Azure text-embedding-3-small
Vector retrieval	FAISS
Generation	Azure gpt-5-mini-1
RAG orchestration	LangChain RetrievalQA
Source visibility	return_source_documents=True
Secret handling	.env with python-dotenv

12. Recommended Improvements

Retrieval evaluation: Create a small test set of questions with expected source documents and measure whether the correct chunk appears in the top results.

More targeted test questions: Use specific hotel names, policy names, or entities rather than broad questions such as 'best hotels'.

Metadata visibility: Print source filename and page number with each retrieved chunk.

Chunking experiments: Compare different chunk sizes, overlaps, and splitters.

Retriever tuning: Experiment with top-k retrieval and metadata filters.

Grounding prompt: Require the LLM to answer only from retrieved context.

Persistence: Save the FAISS index so it does not need to be rebuilt every session.

13. Suggested Evaluation Questions

Question	Expected Source
What is the return policy?	faq.txt
How much checked baggage can I bring?	Travel PDF - Baggage Guidelines
When is the final trip payment due?	Travel PDF - Payment Methods
Does Mountain Pine Lodge allow pets?	Hotel Booking PDF - Mountain Pine Lodge
What is the parking fee at Harbor View Hotel?	Hotel Booking PDF - Harbor View Hotel
What time is standard hotel check-in?	Hotel Booking PDF / Travel PDF depending on wording

14. Final Conceptual Summary

The uploaded notebook implements an end-to-end multi-document RAG system. Custom TXT and PDF files are ingested, transformed into LangChain Document objects, chunked into smaller semantic units, converted to vector embeddings, indexed in FAISS, retrieved by similarity, and passed to an Azure OpenAI chat model for generation.

The implementation successfully demonstrates the core engineering pattern behind RAG systems. The next stage of maturity is not adding more code, but improving retrieval quality, grounding, metadata, evaluation, and operational reliability.

Architecture to remember:

Load -> Chunk -> Embed -> Store -> Retrieve -> Generate

Prepared from the uploaded RAG notebook report and its recorded outputs.